Smart Greenhouse Management System: Phase 4 (Observability & Monitoring)

Source Code Repository: greenhouse-management-dsa-lab

1. Executive Summary

This document outlines Phase 4 of the Smart Greenhouse Management System, focusing on system observability. By integrating Prometheus for time-series metrics and Grafana for visualization, this phase provides deep insights into the operational health, traffic patterns, and performance bottlenecks of the distributed architecture. Additionally, centralized logging was implemented to correlate performance metrics with application logs in a single pane of glass.

2. Instrumentation Strategy (Metrics & Labels)

To effectively monitor both layers of the application, distinct metrics were exposed using the Prometheus client libraries.

- **Metrics Tracked:**
  - **REST API Layer:** `http_requests_total` (Counter) to track throughput, and `http_request_duration_seconds` (Histogram) to track latency.
  - **gRPC Backend Layer:** `grpc_requests_total` (Counter) and `grpc_request_duration_seconds` (Histogram) to monitor internal RPC calls.
- **Dimensional Labels:** Metrics are tagged with `method` (e.g., GET, POST), `endpoint` (e.g., `/items`), and HTTP/gRPC `status` codes.
- **Low-Cardinality Design:** To prevent time-series database exhaustion (crashing Prometheus), raw dynamic URLs (which contain unique UUIDs) are never logged directly as labels. Instead, a `route_pattern()` function is utilized to group all dynamic requests under finite, static route rules (e.g., normalizing `/items/12345` to `/items/<string:item_id>`).

3. Prometheus Configuration

Prometheus was configured via `prometheus.yml` to actively scrape telemetry data from the internal Docker Compose network. The defined scrape targets are:

- `rest-service:5000` (Scraping Flask/Traefik metrics)
- `grpc-service:9103` (Scraping backend gRPC metrics)

4. Load Testing Observations

A load-generation script (`generate_load.py`) was utilized to simulate a burst of traffic containing a mix of valid and invalid requests. Real-time analysis via the Grafana dashboard yielded the following insights:

1. **Traffic Distribution:** The `GET` endpoint received the highest volume of traffic. The REST request rate panel indicated `GET` requests peaking at over 3 requests per second, whereas `POST` requests peaked at approximately 1 request per second.

2. **Error Tracking (4xx/5xx):** Deliberate client-side (4xx) errors were successfully registered. The Error Ratio panel (tracking `status=~"4.."`) spiked accordingly, and the logging panels explicitly captured multiple `GET /health HTTP/1.1" 404` entries.
3. **Latency Impact:** Latency visibly degraded under active load. The REST p95 Latency panel climbed steadily (from ~0.090 seconds up to ~0.097 seconds), indicating that the system slows down slightly as it processes concurrent `POST` and `GET` requests.
4. **REST to gRPC Correlation:** When the REST service receives a `POST /items` request, it translates this into the internal `AddItems` gRPC method. The "gRPC Calls" panel clearly displayed a corresponding spike for `{method="AddItems", status="OK"}` synchronized with the REST POST traffic.

5. Curiosity Extension: Centralized Logging

- **Implementation:** Extended the observability stack by deploying **Loki** (log aggregation) and **Promtail** (log shipping agent).
- **Key Learnings:** By provisioning Loki as an additional data source in Grafana and mapping `promtail` to the Docker socket, I achieved true observability correlation. I was able to observe

metric spikes (e.g., latency or request rates) and immediately cross-reference them with container logs in the same UI. For instance, I successfully traced the exact `Persisted plant/bed to DB` log outputs from the `grpc-service` executing concurrently alongside the tracked metrics.